Introduction

This book is not quite what it looks like.

It looks like a book about AI: about agents, specs, workflows, and the changing shape of software development. Those things are in here. But they're not what the book is about.

What the book is about started with a conference talk, a university friend, and 18 PlayStation Move controllers.

We were building a demo for KubeCon. The talk was called "18 Bluetooth Controllers Walk Into a Bar," about observability and runtime configuration for JoustMania, an open-source party game where players jostle motion controllers until someone falls over. Complex execution: multiple Bluetooth adapters, battery-powered devices, sensors firing at 100Hz. When a player complains their controller "felt different," how does anyone debug it at 2am at a convention?

A good and genuinely interesting problem and two people who knew the domain and cared about the project.

As we had different schedules, we started hacking on our own. We both made progress and moved fast.

And we never quite planned it together.

The knowledge gap opened quietly. Decisions got made that the other person didn't know about. Assumptions turned out not to be shared. Work that should have built on itself didn't quite fit together. Not because either of us was wrong, but because we hadn't stopped to build the shared picture before we started building the thing.

The fix wasn't better tools, a different framework, or a smarter approach to Bluetooth telemetry. What we'd skipped was simpler and harder than any of that.

The book is about the conversation we didn't have.

What happened between me and my university friend is happening to software teams everywhere right now, at a scale and speed that makes the gap much harder to notice and much more expensive to close.

AI gave every engineer a tool that makes starting to hack immediately feel productive. The agent is ready. The prompt is right there. Why stop to talk? Why slow down for the conversation? The output is coming, it looks right, the ticket will close.

And somewhere in the gap between all that individual motion, the understanding stops being shared.

There's a structural reason AI has this problem.

A model generates with the same confidence whether the output is right or wrong. The checks that catch it, the tests, the linters, the review, sit around the model, not inside it. Strip them away and nothing changes in the generation. No peer review. No colleague who reads it and says "that's not how it works."

The same thing happens to engineers who go heads down with AI tools and stop talking to each other. The output keeps coming, the confidence stays high, and the errors accumulate quietly, until someone external catches them, or until nobody does.

AI hallucination and team misalignment aren't the same mechanism, but they rhyme: both produce confident output that nobody is checking against shared intent.

The industry's response to AI has been, mostly, to go faster.

Add the tool. Ship more. Reduce the headcount that feels redundant when the agent can implement. Optimize the individual. Measure the output. Skip the room.

That's a familiar move. The industry has cut the deliberate conversation before, rationally, because the process it had was tedious and wasn't bringing value — and learned, sometimes painfully, that the conversation was the work. How that happened, and how AI is offering the same deal at a larger scale, is the Phoenix's story to tell.

Extreme Programming understood what was at stake in the nineties.[1] Mob programming, pair programming, collective ownership: an entire practice built on the conviction that software development is a social activity. The understanding built through collaboration was the point.

[1] Kent Beck, *Extreme Programming Explained: Embrace Change* (Addison-Wesley, 1999).

The industry moved on. Faster frameworks. Individual metrics. Optimized handoffs. The team became a coordination cost to minimize.

The industry is rediscovering, under pressure, what XP already knew.

It begins with Sisyphus, and with me as the problem, pushing harder against a process that was already broken and shipping faster with AI without ever changing the terms of the curse.

It ends with the Phoenix, the choice to rise differently, a rediscovery of what was always true in a new form.

In between, it follows one thread, shared understanding, through its whole life. Where it belongs, what it actually is, and who builds it. What it's worth once implementation gets cheap. How a team creates it together, challenges it until it holds, and keeps it from dying when the people who held it move on. How you tell it's really there, what structure and culture keep it alive, and what it takes to scale it across an organization without thinning it to nothing.

The thesis: AI erodes the incidental friction that used to produce shared understanding as a byproduct of the work. That makes shared understanding the scarce resource, and constructive friction the mechanism that protects and tests it. Implementation getting radically cheaper is why this is happening now, and at scale. The organizations that preserve the constructive kind will outperform the ones that optimize it away.

This has happened before, and the shape repeats. Assembly gave way to compilers and the hard part moved from getting the instructions right to designing the program. Frameworks absorbed the plumbing and the hard part moved to modeling the domain. Cloud absorbed the servers, open source absorbed whole categories of code, and each time the scarce thing moved further from the machine and closer to intent. Agents are the largest instance of that move, not a departure from it. Which is why the argument here should outlast the tooling that prompted it: what gets cheap changes, and the direction the constraint travels doesn't.[2]

This is a practitioner's book, not an academic argument. It doesn't claim to be the first to notice that teams matter or that shared understanding is valuable. Those ideas have lineage: Peter Naur's theory-building view,[3] XP, domain-driven design,[4] collective code ownership, and

[2]Fred Brooks made the general case in "No Silver Bullet: Essence and Accident in Software Engineering" (1986; reprinted in *The Mythical Man-Month*, anniversary edition, Addison-Wesley, 1995). The accidental complexity of software — the part tooling can remove — keeps falling, while the essential complexity of specifying and designing the conceptual construct does not. This book is an argument about what happens to the essential part when a very large amount of the accidental part disappears at once.

[3]Peter Naur, "Programming as Theory Building" (1985; reprinted in *Computing: A Human Activity*, ACM Press/Addison-Wesley, 1992). Naur argued that a program is the theory its builders hold, that the theory cannot be fully captured in documentation, and that a program whose builders have dispersed is effectively dead.

[4]Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software* (Addison-Wesley, 2003).

an academic literature of their own.[5] What's new is the gap: every current tool and framework for AI-assisted development solves alignment between one person and one agent. None of them address what happens when the spec needs to emerge from a room of people who don't yet share the same understanding. The spec: what to build and why, held in common. Not a document.[6] This book is the argument for that layer, the human and social conditions that make any spec-first approach work.

The conversation that feels slower than prompting alone is doing something the prompt cannot do.

The argument in this book didn't emerge in isolation. While writing it, I kept finding people who'd arrived at the same conclusions on their own. The pressure toward this model is real enough that people keep hitting it without coordinating.

I started writing this book alone, with no one to tell me whether the idea was right. An argument for teams, begun without one, which is why the absence shapes its early chapters. The readers came later, once there was a draft to show.

If you feel like something is wrong but can't point to it, if the speed feels good and the output looks right but something is missing, this book was written for you.

It's written for engineers, but also for the people in the room with them. Product owners trying to understand why AI adoption hasn't made delivery more predictable. Team leads watching their teams drift apart. Stakeholders wondering why the spec keeps getting misunderstood. If you care about how software gets built together, not just how fast it gets built, you're in the right place.

You're not behind. You're seeing what the speed costs.

We started hacking. We forgot to talk to each other. We forgot to build the picture together.

This is the conversation we should have had first.

---

[5] Martin Glinz and Samuel Fricker, "On Shared Understanding in Software Engineering: An Essay," *Computer Science — Research and Development* 30, nos. 3–4 (2015): 363–376.

[6] Throughout this book, *spec* means this shared understanding. Where the formal sense is meant — a contract, like the OpenFeature specification — the full word is used.